CS1000: Introduction to Computing

Lecture: X

Computer Architecture

Abhijit Das
Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Instructor

- **Name:** Abhijit Das
- **Affiliation:**
  - **Current:** Assistant Professor, IIT Hyderabad, India
  - **Past:**
    - Director of Research at UPC BarcelonaTech, Spain
    - Post-Doctoral Researcher at INRIA, France
    - Senior Silicon Design Engineer at AMD Inc.
- **Education:** PhD (Computer Science), IIT Guwahati, 2021
- **Research Interests:**
  - Computer Architecture
  - Systems for AI/ML
  - Quantum Computing Systems
- **Home:** Agartala, Tripura, India



Before Anything Else – A Puzzle

PROGRAM A

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        sum += A[i][j];
```

i then j

PROGRAM B

```
for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        sum += A[i][j];
```

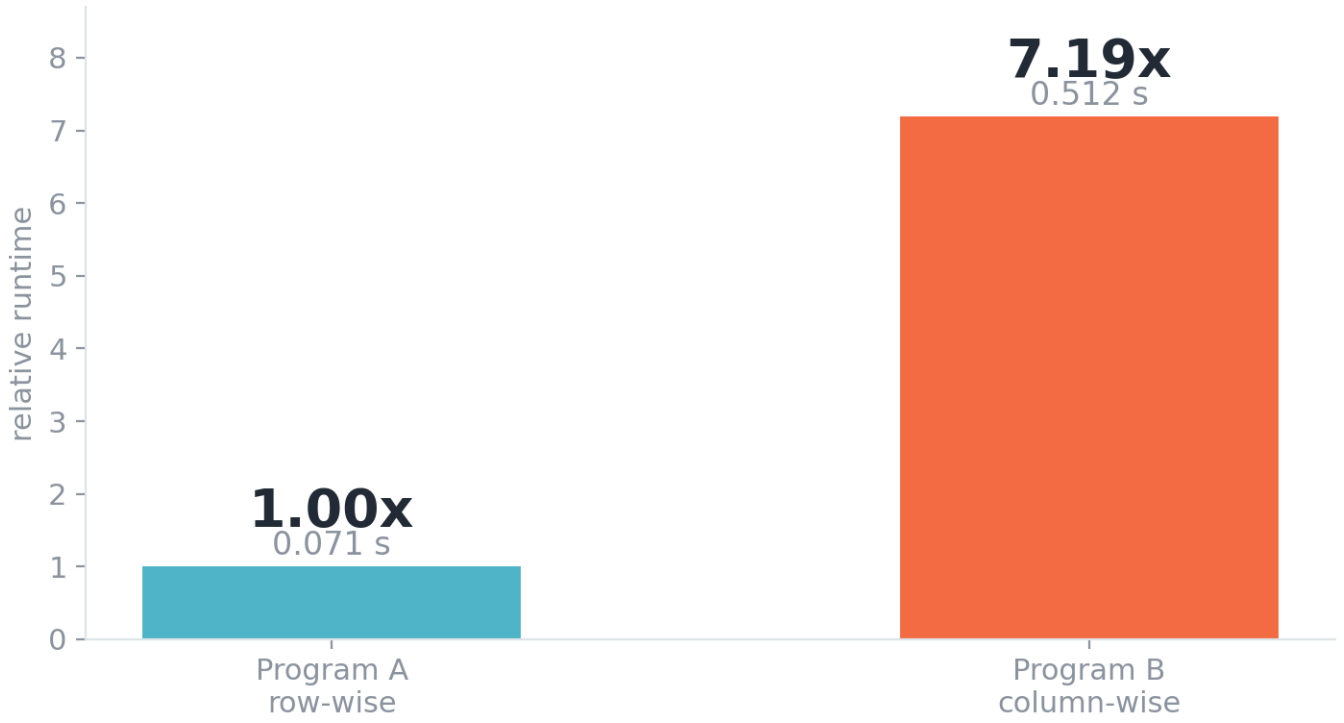
j then i (two variables swapped)

Two loops. Same array. Same arithmetic. Same compiler flags

Which one is faster — and by how much?

Same Algorithm. Same Instructions. 7× Apart

N = 8192 (268 MB), single core, -O2



The compiler did nothing wrong

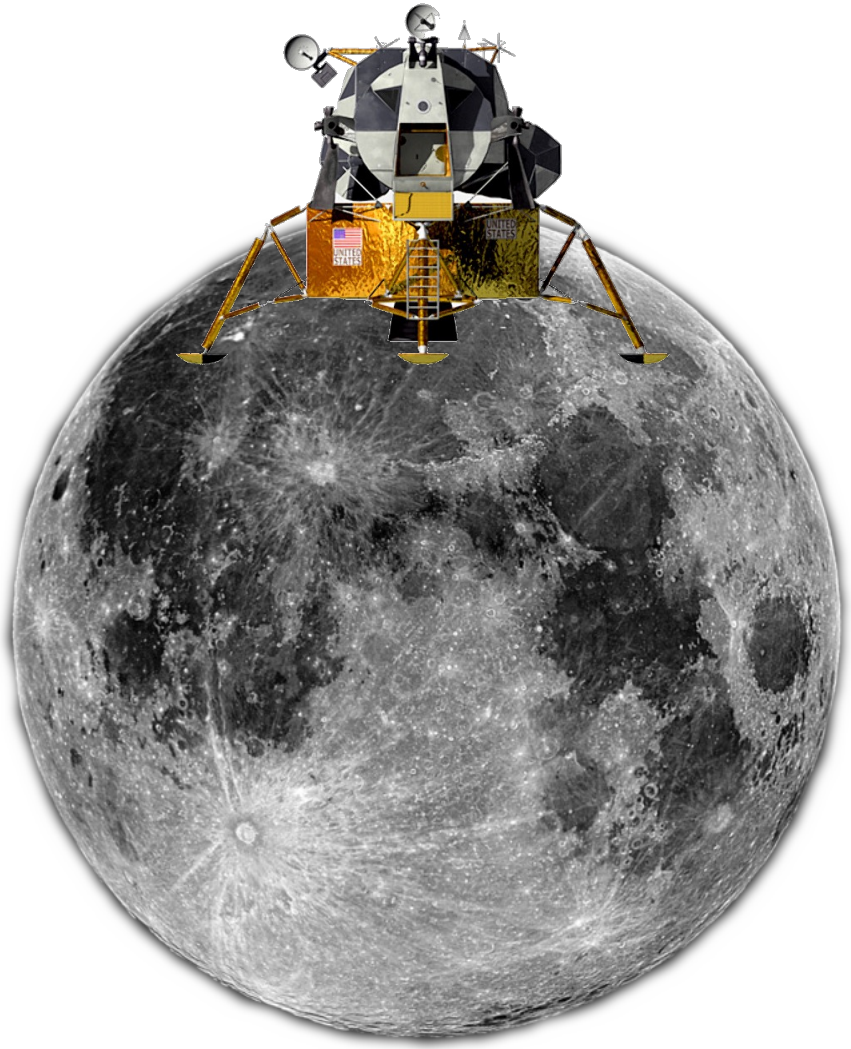
The algorithm is identical

The hardware is identical

The only difference is the ORDER in which the program touches memory

We will come back to why

How Powerful is Your Mobile Phone?



100,000x <



Source: https://www.independent.co.uk/ 2019/Apollo 11 anniversary: Could an iPhone fly me to the moon?

You are Carrying a Supercomputer

~10 B	~3 B/s	~5 W	99.9%
Transistors in a phone	Instructions executed	Power less than a lamp	Machinery you don't know

- Every app you love is a thin layer of text sitting on top of this
- Computer architecture is the study of the thing underneath
  - How a machine made of switches ends up running *Instagram*

Nobody Understands the Whole Computer

Applications	The app you open
Programming Languages	C, Python, Java
Operating System	Linux, Android
Instruction Set Architecture	The hardware/software contract
Microarchitecture	How the contract is actually kept
Logic Gates	AND, OR, NAND
Transistors & Physics	Electrons in silicon

- Each layer promises something to the layer above, and hides how it keeps that promise
  - *Break the promise and every layer above collapses*
- **Computer Architecture lives in the two highlighted layers**

Chasing One Line of C

c = a + b; - what actually happens?

1

C source

```
c = a + b;
```

2

Assembly (ARM)

```
ADD X2, X0, X1
```

3

Machine code

```
1000 1011 0000 0001
...
```

4

Hardware

64 full adders

~600 transistors

Under a nanosecond

- *Nothing here is magic*
- *Every single step is a translation you could do by hand*
  - *and in this course you will*

How Many Switches are We Talking About?

Transistor counts in a single chip

1971		2,300
1993	Intel Pentium	3.1 million
2006	Core 2 Duo	291 million
2020	Apple M1	16 billion
2024	NVIDIA Blackwell GPU	208 billion

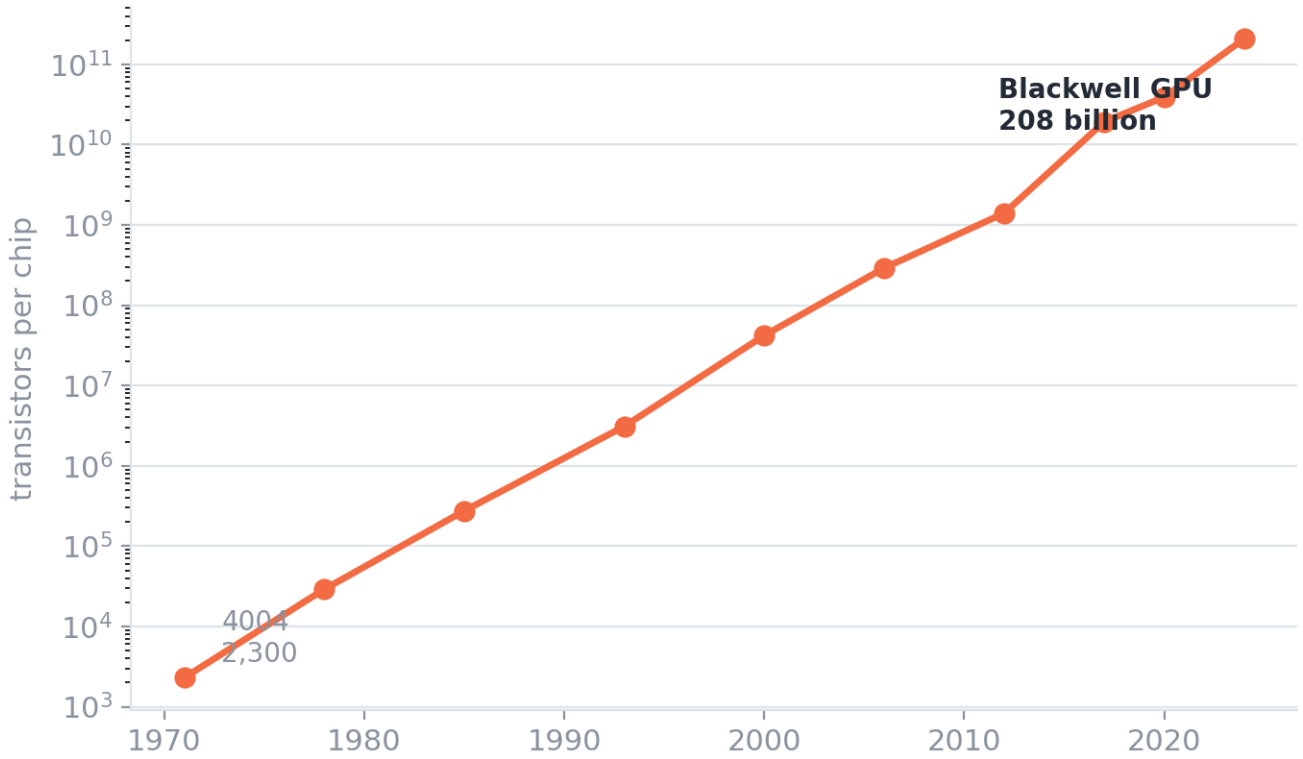
208,000,000,000

If each transistor were a grain of rice, one chip would weigh about 5,000 tonnes – roughly 200 shipping containers of rice!

Every transistor has to work. Every single time

The Graph that Built Our World

Transistors per chip, 1971 → 2024 (log scale)

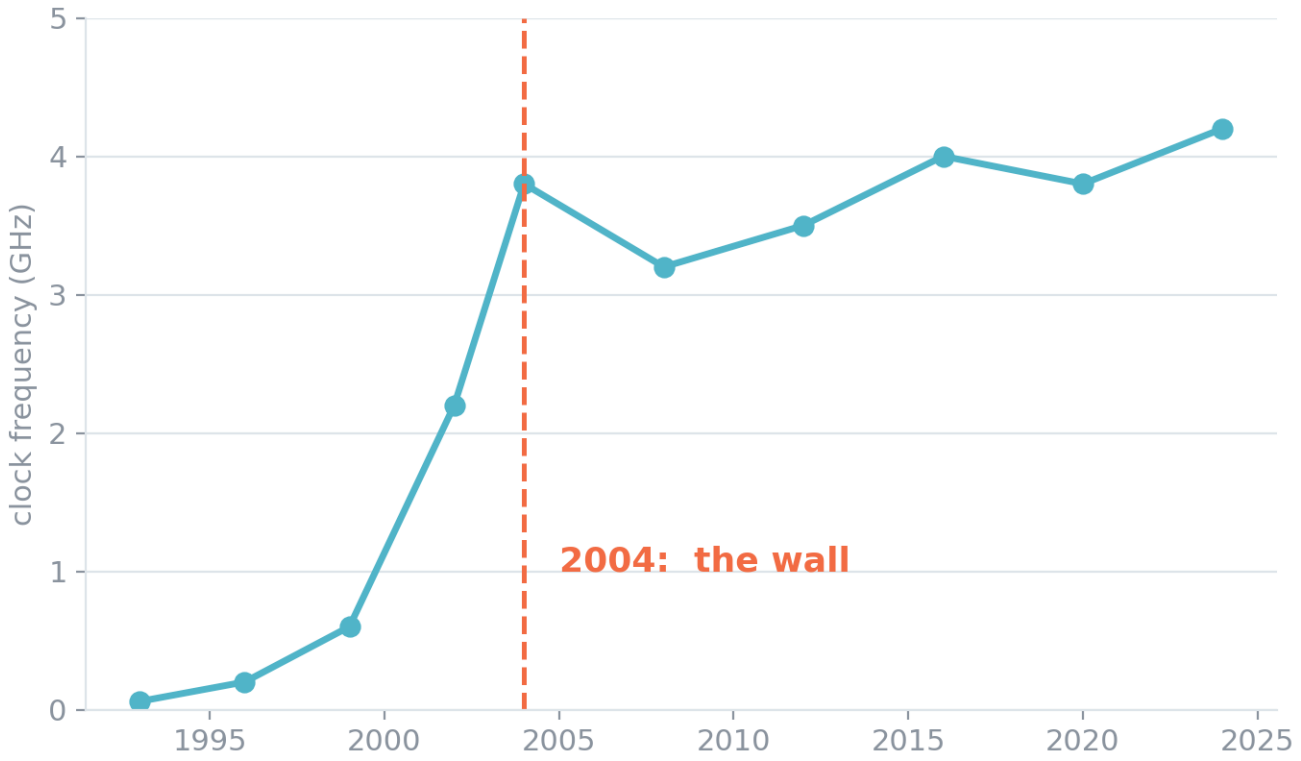


Moore's Law

- 1965: Gordon Moore notices that transistor counts double roughly every two years
- It was never a law of physics, but an industry deciding to keep a promise
- It held for fifty years – the only exponential we have sustained that long

And The Graph Nobody Shows You

Clock frequency of a top desktop CPU



2004

The year the speedometer stopped

- Your 2024 processor runs at about the same clock speed as a 2004 Pentium 4
- **And yet it is perhaps 50× faster**
- Everything that happened in those twenty years is computer architecture

Why it Stopped: The Power Wall

Dennard Scaling ended around 2005

Smaller transistors stopped getting proportionally cheaper to run

Power $\propto f$

Push the clock higher and power climbs with it. Push voltage too, and it climbs with the square

Power becomes heat

A chip that kept scaling this way would reach the power density of a rocket nozzle. Silicon melts first

So: go wide, not fast

If you cannot make one core faster, put many on the chip. 2005: the multicore era begins - out of necessity

What architects did instead

2005 Multicore → **2010** Heterogeneity → **2016** Domain-specific chips → **2020** Chiplets & 3D stacking

Illustration: Multi-Core Processor



Source: Images are generated by Microsoft's AI Image Generator

Trick 1: Pipelining

Doing the laundry taught us how to build processors

The Naive Way

Wash a load. Wait. Dry it. Wait. Fold it. Then start the next load.

Three loads take nine time slots. The washing machine sits idle two-thirds of the time.

9 slots

The Pipelined Way

The moment load 1 goes to the dryer, load 2 goes into the washer. Every machine stays busy.

Each load still takes three slots. But loads come out three times as often.

5 slots

A processor pipelines instructions exactly this way

fetch, decode, execute, memory, write-back

Modern cores are 15–20 stages deep with ~200 instructions in flight at once

Trick 2: The Processor Guesses the Future

and when the guess is wrong, it costs 15–20 cycles

```
if (data[i] >= 128)
    sum += data[i];
```

The pipeline is 20 stages deep; the outcome is not known for another 15. Waiting is unaffordable – so the processor guesses, and executes the guess

A real, famous question

"Why is processing a SORTED array six times faster than an unsorted one?"

Same data. Same code. Same total work

And then the guessing turned into a security hole

Spectre & Meltdown, 2018

Speculatively executed code leaves traces in the cache even after the results are discarded

Every processor on Earth

A web page could read passwords out of another program's memory. The fixes cost 10–30% on some workloads

The lesson

Performance and security are not separate design axes. They trade against each other, in hardware

Trick 3: Hiding the Fact that Memory is Slow

If one CPU cycle were one second, here is how long a memory access would

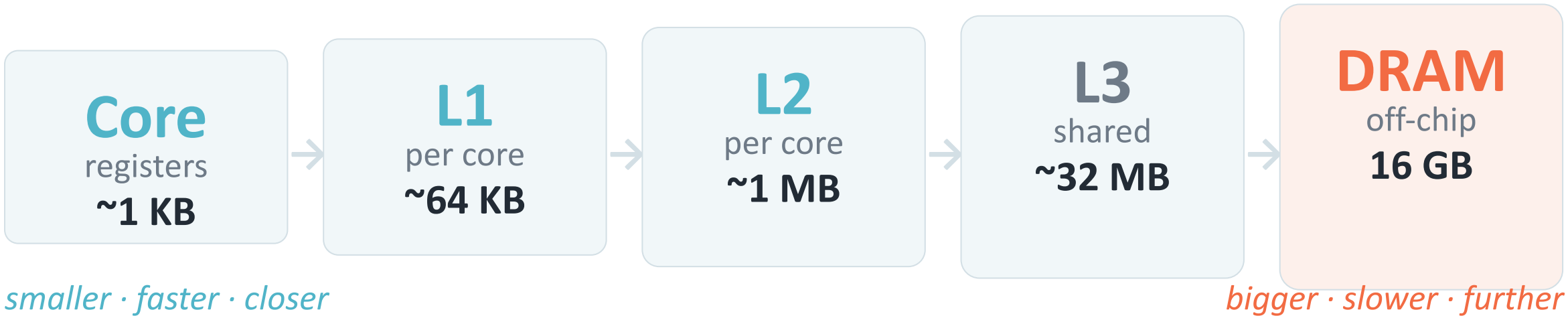
Register	1 cycle	**1 second**
L1 cache	4 cycles	**4 seconds**
L2 cache	12 cycles	**12 seconds**
L3 cache	40 cycles	**40 seconds**
Main memory (DRAM)	~200 cycles	**3.5 minutes**
SSD	~100 µs	**3.5 days**
Hard disk	~10 ms	**1 year**

The memory wall

- Processors got ~1000× faster; memory got ~10× faster
- That gap is the defining problem of modern computer arch
- **A fast processor spends most of its life waiting**

The Fix: Keep What You will Need Close By

A cache is a bet that you will use the same data again – and its neighbours



Crucial Detail: Memory is never fetched one byte at a time. It arrives in 64-byte blocks called cache lines. Ask for one number, and you are handed sixteen

So – The Puzzle, Solved

Program A walks along cache lines. Program B jumps across them

Program A · row-wise

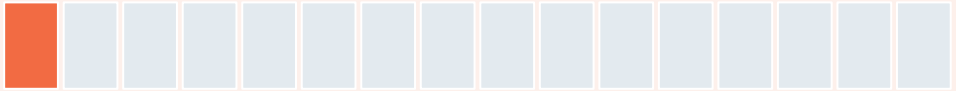


One cache line arrives; all 16 values are used

1 memory trip per 16 elements

efficient

Program B · column-wise



One cache line arrives; exactly 1 value is used, then the line is evicted before the program returns

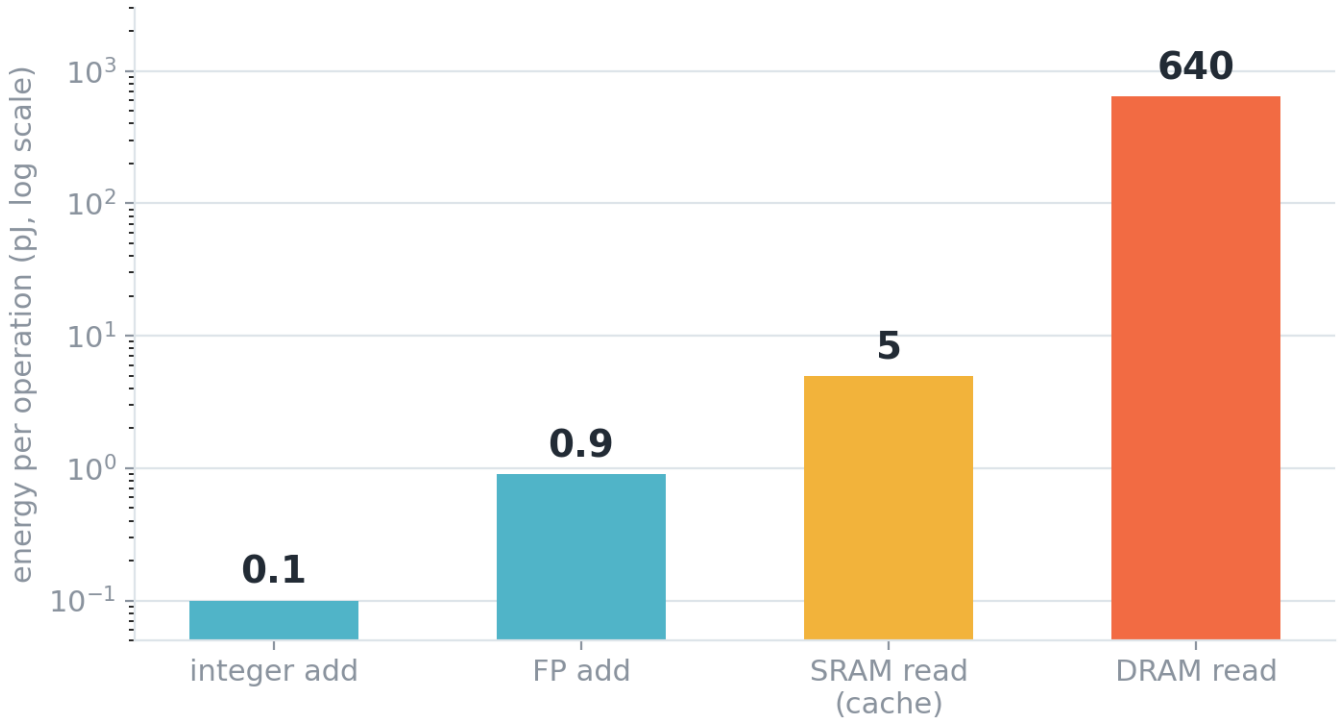
1 memory trip per element

94% of the bandwidth is thrown away

Two variable swapped. Sixteen times more memory traffic. Nothing in the source code tells you this

Trick 4: The One We Have Not Solved Yet!

The energy cost of one 32-bit operation (45 nm, picojoules)



6,400×

- Fetching a number from memory costs thousands of times more energy than doing arithmetic on it
- **Computation is essentially free. Moving data is what you pay for – in energy, in time, and in carbon**

Source: Horowitz, "Computing's Energy Problem", ISSCC 2014 — still the reference numbers a decade later

Where the Field is Right Now

AI did not happen because of algorithms alone

Backpropagation is from 1986. What changed was the machine. And today a modern AI accelerator can do far more arithmetic than it can be fed with data – the expensive matrix engines sit idle, waiting for weights to arrive from memory

1 Bring memory closer

Stack DRAM on top of the logic that uses it. Shorten the wire, save the energy.

2 Compute inside memory

Do the arithmetic in the memory array itself, and never move the data at all.

3 Better on-chip networks

With 100+ cores, the chip is a city and the interconnect is its road system.

4 A stranger machine

Quantum computers need architects too — layout, routing, control — with far harder constraints.

My research group works on some of these

Five Honest Reasons to Care

Even if you never design a chip

1	**It is the last free lunch**	Nobody will hand you a faster clock again. The remaining speedups come from understanding the machine
2	**Every AI advance is an architecture advance**	The models you admire exist because someone built a machine that could feed them
3	**Security lives down here**	Spectre was not a software bug. Some of the worst vulnerabilities of the last decade were hardware features
4	**It makes you fearless**	Once you know what is underneath, nothing in the stack is a black box. Debugging changes character entirely
5	**It is where the frontier is**	Physics has stopped helping. The interesting problems are open – and open problems are where new people get to matter

Three Things You Can Do This Week

None of them require permission, money, or a course prerequisite

1

Reproduce the 7×

Write both loops from slide 3. Time them. Change N until the ratio appears and then disappears – you will have measured your own cache size.

2

Interrogate your machine

Run `lscpu` and `lstopo` on Linux, or `sysctl hw.cachelinesize` on a Mac. Find out how many cores you have and how big each cache is.

3

Break something on purpose

Write a program that misses cache every time. Then fix it. The gap between those two numbers is your first architecture result.

Software is the tenant. Architecture is the building

You may come to my office and show me what you measured

Acknowledgements

- **Dr. Rajesh Kedia**, Dept. of CSE, IIT Hyderabad
  - For sharing insights into how this lecture was conducted in previous years/iterations

- **Claude AI and Microsoft Designer**
  - For assisting with illustrative graphs and images
  - **Notes**:
    - The generated visuals are intended solely for illustration and conceptual explanation
    - They should **not be interpreted as quantitatively accurate, to scale, or as representing actual results**

Thank you!

For further questions, please reach out at

abhijit.das@cse.iith.ac.in

https://www.iith.ac.in/~abhijit.das/

